

Search-FilesForWord.ps1

PowerShell script - recursive substring search for a word/text fragment across files in folders and subfolders.

```
<#
```

```
Search-FilesForWord.ps1
```

```
Searches for a word/text fragment inside files, recursively through subfolders.
The search is a plain substring match (not "whole word"), so it will also match
the term when it appears inside another word or combined with hyphens/underscores,
e.g. searching for "server" will also match "fileserver", "server-01", "my_server", etc.
```

```
For every match it prints the file, the line number, the line itself and how many
times the term appears on that line, followed by an occurrence count per file and
an overall total. Folders and files that could not be opened (typically "access
denied") are not silently skipped - they are listed at the end with the reason.
```

```
USAGE EXAMPLES:
```

```
# Search for "server-01" under the current folder, in all files
.\Search-FilesForWord.ps1 -SearchTerm "server-01"
```

```
# Search under a specific folder
.\Search-FilesForWord.ps1 -SearchTerm "server-01" -Path "C:\Logs"
```

```
# Limit the search to specific file types
.\Search-FilesForWord.ps1 -SearchTerm "server-01" -Path "C:\Logs" -Include *.log,*.txt,*.csv
```

```
# Case-sensitive search
.\Search-FilesForWord.ps1 -SearchTerm "Server-01" -CaseSensitive
```

```
#>
```

```
[CmdletBinding()]
```

```
param(
```

```
# The word / text fragment to search for. Matches as a substring, so hyphens and
# partial-word matches (e.g. inside "fileserver-01") are found as-is.
```

```
[Parameter(Mandatory = $true)]
```

```
[string]$SearchTerm,
```

```
# Root folder to search from. Defaults to the current directory.
```

```
[string]$Path = ".",
```

```
# Optional file name filters (wildcards), e.g. *.txt, *.log, *.csv.
```

```
# Leave empty to search every file.
```

```
[string[]]$Include = @(""),
```

```
# Perform a case-sensitive search. By default the search is case-insensitive.
```

```
[switch]$CaseSensitive
```

```
)
```

```
if (-not (Test-Path -LiteralPath $Path)) {
```

```
Write-Error "Path not found: $Path"
```

```
return
```

```
}
```

```
function Get-OccurrenceCount {
```

```
param(
```

```
[string]$Text,
```

```
[string]$Term,
```

```
[switch]$CaseSensitive
```

```
)
```

```
$comparison = if ($CaseSensitive) { [System.StringComparison]::Ordinal } else { [System.StringComparison]::OrdinalIgnoreCase }
```

```

$count = 0
$index = 0
while (($index = $Text.IndexOf($Term, $index, $comparison)) -ge 0) {
    $count++
    $index += $Term.Length
}

return $count
}

function Show-InaccessiblePaths {
    param($Items)

    if (-not $Items) {
        return
    }

    Write-Host "`nPaths that could not be accessed (skipped): $($($Items).Count)" -ForegroundColor Yellow
    $Items | Format-Table -AutoSize -Wrap
}

# Collect the enumeration errors instead of swallowing them, so folders that
# could not be opened (usually "access denied") can be reported at the end
# rather than silently skipped.
$allFiles = Get-ChildItem -Path $Path -Include $Include -Recurse -File -ErrorAction SilentlyContinue -ErrorVariable $errors

$results = foreach ($file in $allFiles) {
    $selectStringParams = @{
        Path      = $file.FullName
        Pattern    = $SearchTerm
        SimpleMatch = $true
        ErrorAction = "SilentlyContinue"
    }
    if ($CaseSensitive) {
        $selectStringParams["CaseSensitive"] = $true
    }

    Select-String @selectStringParams -ErrorVariable +accessErrors
}

# Folders/files that could not be read, with the reason. TargetObject holds the
# path for these provider errors; fall back to the error text when it is empty.
$inaccessible = $accessErrors |
    ForEach-Object {
        [PSCustomObject]@{
            Path = if ($_.TargetObject) { $_.TargetObject } else { $_.CategoryInfo.TargetName }
            Reason = $_.Exception.Message
        }
    } |
    Where-Object { $_.Path } |
    Sort-Object Path -Unique

if (-not $results) {
    Write-Host "No matches found for '$SearchTerm' under '$Path'."
    Show-InaccessiblePaths -Items $inaccessible
    return
}

# Print every matching line, from the file it was found in, with how many
# times the search term appears on that specific line.
$results |
    Select-Object @{Name = "File"; Expression = { $_.Path } },
        @{Name = "Line"; Expression = { $_.LineNumber } },
        @{Name = "Text"; Expression = { $_.Line.Trim() } },

```

```

        @{Name = "Occurrences"; Expression = { Get-OccurrenceCount -Text $_.Line -Term $SearchTerm -Ca
Format-Table -AutoSize -Wrap

# Count how many times the word was found overall, and per file.
$perFileCounts = $results | Group-Object Path | ForEach-Object {
    [PSCustomObject]@{
        File = $_.Name
        Count = ($_.Group | ForEach-Object { Get-OccurrenceCount -Text $_.Line -Term $SearchTerm -CaseSensitive:$
    }
}

Write-Host "Occurrences per file:"
$perFileCounts | Format-Table -AutoSize

$totalOccurrences = ($perFileCounts | Measure-Object -Property Count -Sum).Sum
Write-Host "`nTotal matching lines: $($results.Count)"
Write-Host "Total occurrences of '$SearchTerm': $totalOccurrences"

Show-InaccessiblePaths -Items $inaccessible

```